

# Task-Specific Goal Costs for Frozen Visual World Models

JEPA Planning Project Team

September 2026

## Abstract

We will take a pre-trained DINO-WM and study its feature space. Specifically, we want to know how we could make use of its learned features and adapt its goal evaluation to a task without training the whole model again. We will freeze the model and train a linear layer, denoted by matrix  $L$ , to compare predicted outcomes with a goal image. Our first task is to push an object to a desired position. The robot’s final hand position does not have to match the hand in the goal image. We will study how many examples we need to teach this distinction, and whether we should collect more object targets or more hand configurations around each target. We want to see whether this helps the planner choose better actions. The linear layer itself is an established method, so we still need to test whether it helps here.

## 1 Problem

We want to use one pre-trained world model for tasks that share a physical environment but have different goals. A world model predicts what may happen after an action. A planner uses these predictions to choose actions. For example, it can consider several ways of pushing an object, predict where each one ends, and choose the outcome that is closest to the goal. In a visual world model, these predictions can be made using image features rather than complete future images [2].

A goal image shows more than the object we want to move. It also shows the robot, the table and the surrounding scene. If our task is to place the object at a target, two photographs can show the same correct placement with different robot hand positions. Comparing every feature equally may give these photographs different scores. Robot-Aware Control already studies the importance of separating the robot from the task in visual control [5]. Here, we will look at what this means for the goal cost of a frozen world model.

This can be done by adding a small adapter for goal evaluation. We choose a linear layer first because it allows an easier investigation of the feature transformation. It can combine feature dimensions and reduce the effect of combinations that do not help us judge this task. Similarity between pre-training and our task gives us a reason to try using the existing features, but it does not establish that a linear layer will be enough. So we will first check whether those features are enough for the comparison we need.

## 2 Basic literature review

JEPA stands for joint-embedding predictive architecture. It learns by predicting encoded features instead of reconstructing every pixel. I-JEPA applies this idea to hidden parts of an image [1]. For planning, DINO-WM uses pre-trained visual features to learn action-conditioned predictions [2]. JEPA-WMs compares choices that affect physical planning [3], while LeWorldModel learns its encoder and dynamics together [4]. Our completed pilot used the released DINO-WM checkpoint from JEPA-WMs. We will keep that checkpoint for the first study so that we can build on a setup we have already tested.

Learning a linear distance is established metric learning [6]. Goal-Contrastive Rewards also learns task scores from goal comparisons [7]. VICReg includes a term that brings paired examples together, while LeJEPA uses SIGReg to constrain the distribution of learned features [8, 9]. We will use these as comparisons when training our head. We will not claim that  $L$  is a new JEPA architecture or that existing regularizers cannot learn the desired behavior. What we want to study is which calibration examples help planning when we have a fixed data budget.

## 3 Goals & Scope

### 3.1 Research scope

We will start with one object-placement task in simulation. Specifically, we want to ask *how should we divide a fixed number of calibration images between different object targets and different robot configurations around each target to improve planning on*

*new targets?* Here, calibration means training the goal-scoring layer. It does not mean calibrating a camera. For now, we will stay with this task and consider a second one if the first study shows a useful result.

### 3.2 Dataset

We will use the Push task in Meta-World [10]. We will generate groups of goal images in the simulator. Within each group, the object will have the same position and orientation, while the robot hand will appear in different configurations. We will check the simulator state before accepting a pair. Images from other groups will show different object targets. Hand configurations will be sampled independently of the target so that the head cannot use the hand as a shortcut for locating the object.

For this comparison, we will use three training budgets and three ways of dividing the images within each budget:

$$B = MK, \quad B \in \{64, 256, 1024\}, \quad K \in \{2, 4, 8\}. \quad (1)$$

$B$  is the number of training images,  $M$  is the number of object targets, and  $K$  is the number of robot configurations per target. At  $B = 256$ , the allocations are  $(M, K) = (128, 2)$ ,  $(64, 4)$  and  $(32, 8)$ . More targets may help the head distinguish nearby destinations. More robot configurations may help it recognize that the same destination can appear in different photographs. We do not yet know which use of the budget will be more helpful.

We will hold out entire target groups and robot configurations for evaluation. Validation and test images will be additional to  $B$  and reported separately. The head will receive labels saying whether two photographs have the same object goal. It will not receive object coordinates. We will use simulator coordinates to verify these labels and to measure physical performance. We will also record how much work it takes to collect these labels.

### 3.3 Models

The base model has an encoder  $E$  and an action-conditioned predictor  $F_\theta$ . The encoder converts an image  $o_t$  into features  $z_t$ . The predictor takes the available feature history  $h_t$  and an action  $a_t$  to estimate the next features:

$$z_t = E(o_t), \quad \hat{z}_{t+1} = F_\theta(h_t, a_t), \quad g = E(o_g). \quad (2)$$

$o_g$  is the goal photograph and  $g$  is its encoded representation. To look further ahead, we repeatedly apply the predictor and update its history with predicted features. We call this repeated prediction a rollout. A candidate  $A = (a_t, \dots, a_{t+H-1})$  is one proposed sequence of  $H$  actions. We write its predicted final features as  $\hat{z}(A)$ .

The pilot checkpoint produces  $N = 256$  image patches with  $d = 384$  features per patch. We will use the same matrix  $L \in \mathbb{R}^{32 \times 384}$  at every patch, giving 12,288 trainable weights. We will apply  $L$  to the predictor’s output and to the encoded goal when computing the cost. We will keep the original features inside the rollout. We will not train the encoder or the predictor. This way, the model keeps its representation of the hand even if the final goal score gives some hand differences less weight.

### 3.4 Compute

We will use the available 32 GB V100 for the first experiments. Other GPUs can be used after checking memory use and runtime. We will request a ceiling of 200 GPU-hours, split into 30 hours for setup and feature extraction, 20 for head experiments, 120 for planning evaluation and 30 for confirmation runs. These are estimates for the work we plan to do. We will also request up to 16 CPU cores, 64 GB of RAM and 300 GB of storage. We do not plan to pre-train a world model.

Most head comparisons can use cached features. Simulator rollouts and repeated world-model prediction are likely to take more time than fitting  $L$ , so we will time a small complete control evaluation before deciding the full run count. The existing pilot ran remotely on the V100 with two numerical CPU threads. No new experiments were run to prepare this document.

## 4 Approach

For each candidate, we will compare the predicted final features with the goal features. We will use the following cost:

$$J_L(A, g) = \frac{1}{N} \sum_{i=1}^N \|L(\hat{z}_i(A) - g_i)\|_2^2, \quad \|L\|_F^2 = 1. \quad (3)$$

The index  $i$  identifies an image patch. Multiplying by  $L$  turns its 384 feature differences into 32 combinations. Squaring and adding these combinations gives the patch cost. We then average across patches to get the candidate’s score. The planner prefers a smaller score. The norm constraint fixes the overall size of  $L$ , so training cannot keep increasing every weight to change the scale of the loss.

We can also write  $\|L\delta\|^2 = \delta^\top W \delta$  with  $W = L^\top L$ . Here  $\delta$  is a feature difference.  $W$  describes how feature combinations contribute to the score. It is positive semidefinite, so the score is never negative. Because  $L$  has only 32 rows, some nonzero differences must receive a zero score. This can help if those differences are irrelevant to the task, but it can hurt if they contain useful object information. So using fewer dimensions is not enough to tell us whether we have learned a useful distance.

To train  $L$ , we will choose an anchor goal  $g$ , another image  $g^+$  with the same object goal, and seven images from other goal groups. We will define their distance using

$$D_L(g, h) = \frac{1}{N} \sum_i \|L(g_i - h_i)\|_2^2, \quad \ell_L(g) = -\log \frac{\exp[-D_L(g, g^+)/\tau]}{\sum_{h \in \{g^+\} \cup \mathcal{N}(g)} \exp[-D_L(g, h)/\tau]}. \quad (4)$$

$\mathcal{N}(g)$  contains the seven different-goal images. The positive number  $\tau$  controls how strongly the loss responds to distance differences. A lower loss means that the same-goal image receives a larger share of the similarity score than the different-goal images. This is a contrastive loss. We need the different-goal examples too, because bringing every image together would leave the planner unable to distinguish destinations.

### Algorithm 1. Training and planning

1. Split target groups into training, validation and test sets. Encode the images once with the frozen encoder.
2. Sample a training group uniformly. Choose two images from it and seven negatives from other groups within the same image budget.
3. Compute Eq. (4). Update only  $L$  by gradient descent, then divide it by its Frobenius norm. Reject a zero-norm update. Use the same number of updates for each allocation.
4. Freeze  $L$ . Predict candidate outcomes with the original world model and score them using Eq. (3).
5. Use the same cross-entropy method (CEM) planner for every cost. CEM samples action sequences, keeps low-cost sequences, and draws new sequences around them [11]. Execute a short part of the chosen sequence, observe the new state, and plan again.

During training, we change the entries of  $L$ . During planning, we change the proposed actions while  $L$  stays fixed. A useful distance between encoded photographs may still fail on predicted features, so we will test both to see whether the improvement carries over to planning.

## 5 Validation

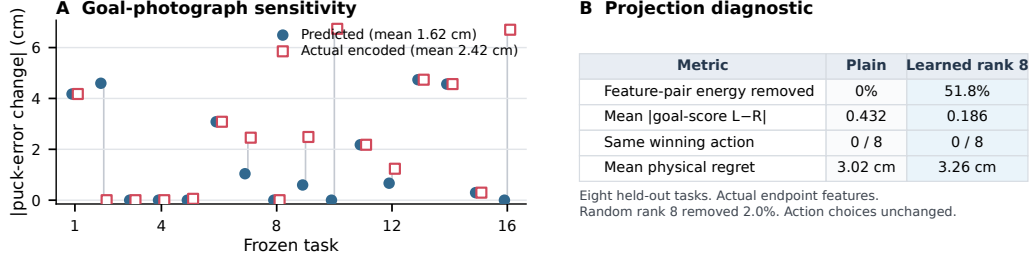
We started by checking how much the goal photograph changes the model’s choice. We tested 16 development tasks with 128 candidate sequences per task, giving 2,048 simulated branches. We then changed the photographed hand configuration while keeping the left and right goal photographs’ object poses identical. With predicted endpoint features, this changed the selected candidate in 10 of 16 tasks and changed the native success outcome in 5 of 16. With actual endpoint features, the counts were 12 of 16 and 7 of 16. Actual endpoints remove prediction error from this comparison, but they are unavailable to a deployed planner.



Seed 58003. Identical puck state. Hand separation = 14.0 cm.

**Figure 1.** Two recorded goal photographs from one pilot task. The object has the same position and orientation. The hand positions are about 14 cm apart. Robot movement also changes shadows and surrounding appearance, so this is not a test of robot pixels alone.

We also tried removing eight feature directions estimated from equivalent goal pairs. On eight held-out development tasks, this removed 51.8% of the paired feature variation. It did not improve agreement on the winning action. With actual endpoint features, mean physical regret changed from 3.02 cm to 3.26 cm. So this correction did not establish an improvement, even though it removed much of the feature variation. We will need to check the decisions themselves when evaluating the learned head. The proposed contrastive head and data-allocation study have not yet been run.



**Figure 2.** Left, each horizontal position is one task. Height shows the absolute change in selected physical endpoint error after changing the goal photograph. Right, removing feature variation did not establish better decisions on the eight held-out development tasks. These are pilot results, not a benchmark replication or a result for the proposed  $L$ .

We will compare the learned head with ordinary feature distance, repeated random 32-dimensional projections, a fixed coordinate subset, and the same head trained with shuffled pair labels. These comparisons will help us see whether any improvement comes from task information or simply from compression. We will also compare a nonlinear head and alternative training losses using VICReg or pair agreement with SIGReg. A coordinate readout and an object-mask cost will provide stronger comparisons, with their additional supervision reported. We will test inexpensive feature comparisons first and take only the strongest methods into full control evaluation.

For a candidate  $A$ , let  $p_H(A)$  be the object’s final position in the simulator and  $q$  its position in the goal photograph. We will measure

$$c_q(A) = \|p_H(A) - q\|_2, \quad \text{Regret}_q(g) = c_q(\hat{A}(g)) - \min_{A \in \mathcal{A}} c_q(A). \quad (5)$$

$\hat{A}(g)$  is the selected candidate and  $\mathcal{A}$  is the shared candidate set. Regret measures how much worse our choice is than the best sequence in that set. It does not compare against an optimal controller. We will also report success within 1, 2 and 5 cm, native simulator success separately, and complete planning time. The earlier pilot used the native simulator target, which is not always exactly the object position shown in the photograph.

We will first score actual endpoints, then predicted endpoints, and finally run control with repeated observation and planning. The final comparison will use 100 held-out task seeds, two equivalent goal photographs per task, and three calibration-data resamples for learned heads. We will keep the planner budgets and executed action lengths the same across comparisons. We will estimate uncertainty across tasks, keeping related photographs together [12]. We will also move the object goal slightly to check that the score still responds when the desired placement changes.

## 6 Deliverables

We will release the goal-image groups, split definitions, trained heads, evaluation code and measured compute costs in the [project repository](#). The report will show how physical performance changes with the number and type of calibration images. We will call this a data-efficiency result unless the evidence supports a stronger scaling claim. We hope to see that a small amount of task supervision improves planning beyond the compression controls. If it does not, we will report which assumption failed and which simpler comparison worked better.

## 7 Timelines & Milestones

| Weeks | Reading and implementation                                                         | Milestone                                                              |
|-------|------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| 1–2   | I-JEPA, DINO-WM and JEPA-WMs [1, 2, 3]. Trace model inputs and validate replay.    | Repeat the goal-photo intervention.                                    |
| 3–4   | Metric learning, VICReg and LeJEPA [6, 8, 9]. Fit heads and compression controls.  | Check whether the head helps with predicted outcomes before expanding. |
| 5–6   | Robot-Aware Control, GCR and iCEM [5, 7, 11]. Integrate matched planning.          | Fix thresholds, splits and runtime measurements.                       |
| 7–10  | Run allocation comparisons and new-target tests. Revisit evaluation practice [12]. | Check whether the improvement remains when actions are executed.       |
| 11–14 | Confirm the result. Add a second task only if justified.                           | Audit failures, release artifacts and write the report.                |

We will divide the work across model integration, data and head training, planning evaluation, and analysis. Each part will be checked by another team member. Roughly half our time will go to reading and small reproductions, with the other half spent implementing and evaluating the study. By week 4, we want to know whether the head improves decisions based on predicted features. If it does not, we will spend time understanding why before adding more methods. By week 8, we also want to see an improvement when the actions are actually executed. If that is still not working, we will stay with the first task and look into what is going wrong.

## 8 Why could AI not do this

Fitting a linear head, wiring a contrastive loss, and calling a released checkpoint inside a planner are coding tasks, and we expect AI tools would be able to implement many of these features. This does not remove the research content, because the implementation is not where the difficulty lies.

The result is not determined by the code. Our question is which calibration images improve planning at a fixed budget. The answer depends on what the frozen DINO-WM encoder happens to represent, and that is a property of a trained model rather than of our program. Even if the code is correct, it doesn’t tell us whether object position survives in those features, whether a linear transformation can isolate it, or whether more object targets help more than more hand configurations.

The design decisions are the important part of this project. Freezing the encoder and predictor so that differences are attributable to the cost. Sampling hand configurations independently of the target so the head cannot use the hand as a shortcut. Holding out entire target groups rather than individual images. Testing on predicted endpoints as well as encoded photographs. Including shuffled-label and random projection conditions so that compression is not mistaken for learning. Additionally deciding on whether the cost function is learned, if we use negative sampling or regularized methods are all decisions that need to be made. An AI asked to build a goal-conditioned cost would not produce each of these and would not know which to keep when they conflict.

## Bibliography

These 12 papers support the model choice, task, training comparisons and evaluation. We have not reproduced every method.

- [1] Assran et al. Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture. 2023.
- [2] Zhou et al. DINO-WM: World Models on Pre-trained Visual Features Enable Zero-shot Planning. 2024.
- [3] Terver et al. What Drives Success in Physical Planning with Joint-Embedding Predictive World Models? 2025, revised 2026.
- [4] Maes et al. LeWorldModel: Stable End-to-End Joint-Embedding Predictive Architecture from Pixels. 2026.
- [5] Hu et al. Know Thyself: Transferable Visual Control Policies Through Robot-Awareness. ICLR 2022.
- [6] Weinberger and Saul. Distance Metric Learning for Large Margin Nearest Neighbor Classification. JMLR 10, 207–244. 2009.
- [7] Biza et al. On-Robot Reinforcement Learning with Goal-Contrastive Rewards. ICRA 2025.
- [8] Bardes, Ponce and LeCun. VICReg: Variance-Invariance-Covariance Regularization for Self-Supervised Learning. ICLR 2022.
- [9] Balestrierio and LeCun. LeJEPA: Provable and Scalable Self-Supervised Learning Without the Heuristics. 2025.
- [10] Yu et al. Meta-World: A Benchmark and Evaluation for Multi-Task and Meta Reinforcement Learning. 2019.

[11] Pinneri et al. [Sample-efficient Cross-Entropy Method for Real-time Planning](#). 2020.

[12] Agarwal et al. [Deep Reinforcement Learning at the Edge of the Statistical Precipice](#). 2021.